

Name: Vakkalanka Sai Bhaskara Sujit

ID number: 2520090085

Week 3

SESSION-5

Task 1: To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

CODE:

GNU nano 8.7.1

```
#include <stdio.h>
#include <string.h>

#define MAX_INPUT 200
#define MAX_QUOTES 20

int main() {
    char input[MAX_INPUT];
    char quoted_strings[MAX_QUOTES][MAX_INPUT];
    int quote_count = 0;

    printf("Session 5 - Single Quote Parsing\n");
    printf("Enter a command with single quotes: \n");
    printf("myshell> ");
    if (fgets(input, sizeof(input), stdin) == NULL) return 0;

    /* Remove newline */
    input[strcspn(input, "\n")] = '\0';

    int i = 0;
    /* Parse input and find single-quoted strings */
    while (input[i] != '\0') {
        if (input[i] == '\\') {
            i++;
            int j = 0;
            /* Store everything until the closing quote */
            while (input[i] != '\0' && input[i] != '\\') {
                if (j < MAX_INPUT - 1) {
                    quoted_strings[quote_count][j++] = input[i];
                }
                i++;
            }
            /* Check for closing quote */
            if (input[i] == '\\') {
                quoted_strings[quote_count][j] = '\0';
                if (quote_count < MAX_QUOTES - 1) {
                    quote_count++;
                }
                i++;
            } else {
                printf("Error: Unclosed single quote.\n");
                return 1;
            }
        } else {
            i++;
        }
    }
}
```

```

        }
        i++;
    } else {
        printf("Error: Unclosed single quote.\n");
        return 1;
    }
} else {
    i++;
}
}

printf("\nQuoted strings found:\n");
if (quote_count == 0) {
    printf("No single-quoted strings found.\n");
} else {
    for (int k = 0; k < quote_count; k++) {
        printf("Quoted String %d: [%s]\n", k + 1, quoted_strings[k]);
    }
}

printf("\nParsing validation:\n");
if (quote_count > 0) {
    printf("Single-quote parsing successful.\n");
} else {
    printf("No quoted content to validate.\n");
}

return 0;
}

```

OUTPUT:

```

sujit@sujit-LOQ-15IRX9: ~
sujit@sujit-LOQ-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills$ nano main.c
sujit@sujit-LOQ-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills$ gcc main.c -o main
./main
Session 5 - Single Quote Parsing
Enter a command with single quotes:
myshell> 'hello world'

Quoted strings found:
Quoted String 1: [hello world]

Parsing validation:
Single-quote parsing successful.
sujit@sujit-LOQ-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills$ ./main
Session 5 - Single Quote Parsing
Enter a command with single quotes:
myshell> hello @world

Quoted strings found:
No single-quoted strings found.

Parsing validation:
No quoted content to validate.
sujit@sujit-LOQ-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills$ █

```

Task 2: To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

CODE:

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 300
#define MAX_TOKENS 50
#define MAX_TOKEN_LENGTH 100

/* Expand environment variables such as $USER */
void expand_variable(char *input, char *output) {
    int i = 0;
    int j = 0;
    while (input[i] != '\0' && j < MAX_INPUT - 1) {
        if (input[i] == '$') {
            i++;
            char variable[50];
            int k = 0;
            /* Read variable name */
            while (isalnum((unsigned char)input[i]) || input[i] == '_') {
                if (k < 49) {
                    variable[k++] = input[i];
                }
                i++;
            }
            variable[k] = '\0';

            /* Get environment variable value */
            char *value = getenv(variable);
            if (value != NULL) {
                int v = 0;
                while (value[v] != '\0' && j < MAX_INPUT - 1) {
                    output[j++] = value[v++];
                }
            }
            else {
                output[j++] = input[i++];
            }
        }
        output[j] = '\0';
    }
}

int main() {
    char input[MAX_INPUT];
    char expanded[MAX_INPUT];
    char tokens[MAX_TOKENS][MAX_TOKEN_LENGTH];
    int token_count = 0;
    int i = 0;
```

GNU nano 8.7.1

```
    }
    i++;
}
if (input[i] == '"') {
    i++;
} else {
    printf("Error: Unclosed double quote.\n");
    return 1;
}
tokens[token_count][j] = '\0';
token_count++;
} else {
    /* Normal token */
    while (input[i] != '\0' && !isspace((unsigned char)input[i]) && input[i] != '"') {
        if (j < MAX_TOKEN_LENGTH - 1) {
            tokens[token_count][j++] = input[i];
        }
        i++;
    }
    tokens[token_count][j] = '\0';
    if (j > 0) {
        token_count++;
    }
}
}

/* Display tokens before expansion */
printf("\nTokens before variable expansion:\n");
for (int k = 0; k < token_count; k++) {
    printf("Token %d: [%s]\n", k + 1, tokens[k]);
}

/* Variable expansion */
printf("\nVariable expansion:\n");
for (int k = 0; k < token_count; k++) {
    expand_variable(tokens[k], expanded);
    printf("Token %d: [%s]\n", k + 1, expanded);
}

/* Validation */
printf("\nParsing validation:\n");
printf("Double-quote parsing successful.\n");
printf("Spaces inside quotes were preserved.\n");
printf("Variable expansion was processed.\n");

return 0;
}
```

OUTPUT:

```
Parsing validation:
No quoted content to validate.
sujit@sujit-L0Q-15IRX9:~/Documents/GitHub/OSSP_2520090085/skills$ nano main2.c
sujit@sujit-L0Q-15IRX9:~/Documents/GitHub/OSSP_2520090085/skills$ gcc main2.c -o main
sujit@sujit-L0Q-15IRX9:~/Documents/GitHub/OSSP_2520090085/skills$ ./main
Session 5 Double Quote Parsing
Double quotes preserve spaces and allow variable expansion.
Enter a command:
myshell> echo "$USER is here"

Tokens before variable expansion:
Token 1: [echo]
Token 2: [$USER is here]

Variable expansion:
Token 1: [echo]
Token 2: [sujit is here]

Parsing validation:
Double-quote parsing successful.
Spaces inside quotes were preserved.
Variable expansion was processed.
sujit@sujit-L0Q-15IRX9:~/Documents/GitHub/OSSP_2520090085/skills$ ./main
Session 5 Double Quote Parsing
Double quotes preserve spaces and allow variable expansion.
Enter a command:
myshell> hello world

Tokens before variable expansion:
Token 1: [hello]
Token 2: [world]

Variable expansion:
Token 1: [hello]
Token 2: [world]

Parsing validation:
Double-quote parsing successful.
Spaces inside quotes were preserved.
Variable expansion was processed.
sujit@sujit-L0Q-15IRX9:~/Documents/GitHub/OSSP_2520090085/skills$ █
```

SESSION-6

Task 1: To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

CODE:

```

GNU nano 8.7.1
#include <stdio.h>
#include <string.h>

#define MAX_INPUT 300

int main() {
    char input[MAX_INPUT];
    char output[MAX_INPUT];
    int i = 0;
    int j = 0;
    int escaped = 0;

    printf("Session 6 Escape Sequence Parsing\n");
    printf("Enter a command with escaped spaces or special characters:\n");
    printf("myshell> ");
    if (fgets(input, sizeof(input), stdin) == NULL) return 0;

    /* Remove newline */
    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0) {
        printf("Empty input.\n");
        return 0;
    }

    /* Process escape sequences */
    while (input[i] != '\0' && j < MAX_INPUT - 1) {
        if (escaped) {
            output[j++] = input[i];
            escaped = 0;
        } else if (input[i] == '\\') {
            escaped = 1;
        } else {
            output[j++] = input[i];
        }
        i++;
    }

    if (escaped) {
        printf("Warning: Escape character at the end of input.\n");
    }
    output[j] = '\0';

    printf("\nOriginal input:\n%s\n", input);
    printf("\nProcessed input:\n%s\n", output);
    printf("\nParser validation:\n");
    printf("Escape sequences processed successfully.\n");
    printf("Escaped characters were preserved.\n");

    return 0;
}

```

OUTPUT:

```
Variable expansion was processed.
sujit@sujit-L0Q-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills$ nano main3.c
sujit@sujit-L0Q-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills$ gcc main3.c -o main1
sujit@sujit-L0Q-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills$ ./main1

Session 6 Escape Sequence Parsing
Enter a command with escaped spaces or special characters:
myshell> ls

Original input:
ls

Processed input:
ls

Parser validation:
Escape sequences processed successfully.
Escaped characters were preserved.
sujit@sujit-L0Q-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills$
```

Task 2: To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching

CODE:

```
GNU nano 8.7.1
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid;
    int status;

    printf("Session 6 - Process Execution\n");

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        /* Child process */
        printf("\nChild Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());
        printf("Executing: ls -l\n");
        execlp("ls", "ls", "-l", NULL);

        /* This runs only if exec fails */
        perror("exec failed");
        exit(EXIT_FAILURE);
    } else {
        /* Parent process */
        printf("\nParent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);
        printf("Parent waiting for child...\n");

        waitpid(pid, &status, 0);

        if (WIFEXITED(status)) {
            printf("\nChild exited with status: %d\n", WEXITSTATUS(status));
        } else {
            printf("\nChild terminated abnormally.\n");
        }
        printf("Parent process completed.\n");
    }

    return 0;
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^_ Replace

^K Cut
^U Paste

^T Execute
^J Justify

^C Location
^/ Go To Line

M-U Undo
M-E Redo

OUTPUT:

```
Escaped characters were preserved.
sujit@sujit-L0Q-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills$ nano main4.c
sujit@sujit-L0Q-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills$ gcc main4.c -o main4
sujit@sujit-L0Q-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills$ ./main4
Session 6 - Process Execution

Parent Process
Parent PID: 29256
Child PID: 29257
Parent waiting for child...

Child Process
Child PID: 29257
Parent PID: 29256
Executing: ls -l
total 1496
-rw-rw-r-- 1 sujit sujit 717983 Aug 20 14:21 'OSSP SWeek-4.docx.pdf'
-rw-rw-r-- 1 sujit sujit 745432 Aug 20 14:53 'OSSP Skill Week2 .docx.pdf'
-rwxrwxr-x 1 sujit sujit 16304 Aug 20 15:09 main
-rw-rw-r-- 1 sujit sujit 1817 Aug 20 15:07 main.c
-rwxrwxr-x 1 sujit sujit 16176 Aug 20 15:11 main1
-rw-rw-r-- 1 sujit sujit 3530 Aug 20 15:09 main2.c
-rw-rw-r-- 1 sujit sujit 1246 Aug 20 15:11 main3.c
-rwxrwxr-x 1 sujit sujit 16344 Aug 20 15:13 main4
-rw-rw-r-- 1 sujit sujit 1164 Aug 20 15:13 main4.c

Child exited with status: 0
Parent process completed.
sujit@sujit-L0Q-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills$
```